

EXAMEN D'ALGORITHMIQUE III L3I

Exercice 1 : listes chaînées (5.0)

Un multi-ensemble est constitué d'un ensemble de couples : (valeur, occurrence).

Chaque couple est représenté par une structure composée de deux membres : valeur (un entier) et son occurrence (un entier positif non nul).

On suppose que le multi-ensemble est représenté par une **liste chaînée doublement chaînée** et dans **un ordre croissant suivant la valeur de l'élément**.

Et Pour faciliter la gestion des multi-ensembles, on rajoutera deux adresses : la **première adresse pour indiquer la tête de la liste et la seconde adresse pour indiquer la queue de la liste**.

On souhaite construire un multi-ensemble, à partir d'une liste d'entiers où certaines valeurs peuvent se répéter.

Exemple : à partir de la liste d'entiers suivante : 13, 4, 1, 2, 13, 1, 0, 9, 0, 19, 12, 13, 1, 13, on obtient le multi-ensemble suivant : { (0, 2), (1, 3), (2, 1), (4, 1), (9, 1), (12, 1), (13, 4), (19, 1) }

Soit la structure ci-dessous représentant un multi-ensemble :

```
typedef struct {
    int val;
    int nbOcc;
}TElementDL;

typedef struct celluleDL{
    TElementDL donneeDL;
    struct celluleDL *suivantDL;
    struct celluleDL *precédentDL;
} *DListe;

typedef struct cellule{
    int taille;
    DListe tete;
    DListe queue;
}MultiEnsemble;
```

- 1) En parcourant le multi-ensemble dans les deux sens ; Ecrire en langage algorithmique (ou C), un algorithme qui détermine l'adresse d'une valeur donnée s'elle existe et NULL s'elle n'existe pas.
- 2) En parcourant le multi-ensemble dans les deux sens ; **écrire en C**, un algorithme qui supprime une valeur donnée dans un multi-ensemble donné.
- 3) Ecrire en langage algorithmique (ou C), **un algorithme qui** construit un multi-ensemble à partir d'une **liste chaînée circulaire** d'entiers donnée.

Exercice 3 : Arbres généraux (3.0 PTS)

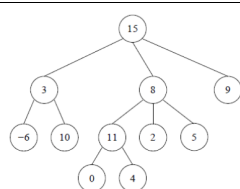
On implémente les arbres généraux de la façon suivante :

```
typedef struct nœud * ArbreG;
struct nœud{
    TElement donnee;
    ArbreG *fils;
};
```

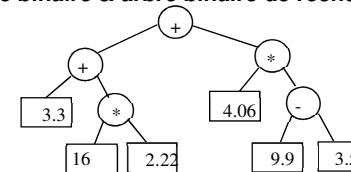
On suppose que tous les nœuds possède le même nombre de fils

- 1) Ecrire en langage algorithmique ou en C, un algorithme qui compte le nombre de feuilles présente dans un arbre donné.
- 2) Ecrire en langage algorithmique ou en C, un algorithme qui effectue le parcours en largeur d'un arbre donné.

Note : on suppose connu, les opérations de la structure intermédiaire.



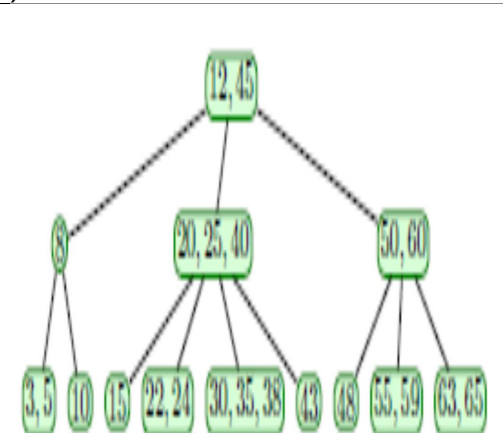
Exercice 2 : arbre binaire & arbre binaire de recherche (7Pts)



- 1) **Ecrire en C**, la structure de donnée pour implémenter sous forme chaînée un arbre binaire, dont le membre **donné** est de type TElement (Réel par exemple).
- 2) **Ecrire en C**, la structure de donnée pour implémenter sous forme chaînée un arbre d'expression (voir exemple ci-dessus), dont le membre **donné** peut être soit un opérateur (un caractère) soit un opérande (un réel).
- 3) **En respectant la structure de l'arbre d'expression, écrire en C**, un algorithme qui détermine l'adresse du plus grand opérande présent dans un arbre binaire d'expression non vide.
- 4) Ecrire en langage algorithmique ou en C, un algorithme d'insertion d'un élément donné, dans un arbre binaire de recherche donné selon le principe suivant :
Si l'arbre est vide, il suffit de créer une feuille contenant l'élément à insérer.
Sinon, on parcourt l'arbre, afin de rechercher un nœud qui sera le père de l'élément que nous désirons insérer.
Une fois le nœud père trouvé, il suffit de créer une feuille contenant l'élément à insérer, et l'attacher du bon côté du nœud père.
- 5) Ecrire en langage algorithmique ou en C, un algorithme qui construit un arbre binaire de recherche formé uniquement des opérandes d'un arbre d'expression donné.
- 6) Appliquer votre algorithme à l'arbre d'expression ci-dessus.

Exercice 4 : Arbre 2-3-4 (5.0 PTS)

- 1) On suppose que les nœuds de l'arbre 2-3-4 sont des 4-Nœuds. Ecrire **en C**, les structures de données nécessaires pour représenter les arbres 2-3-4 dont le membre donné est de type TElement (Entier par exemple).
- 2) Insérer l'élément 27 dans l'arbre 2-3-4 ci-contre.
- 3) Ecrire en langage algorithmique ou en C, le type de nœuds qui apparaît le plus dans un arbre 2-3-4 donné non vide.
- 4) Par le biais d'un arbre 2-3-4, **écrire en C**, l'algorithme de tri d'un tableau composé de « n » éléments (entier par exemple). **On suppose connu, l'algorithme d'insertion d'un élément donné dans un arbre 2-3-4 donné.**



- 5) Prouver la propriété suivante : Un arbre 2-3-4 de hauteur h et contenant « n » nœuds vérifie l'inégalité suivante : $\log_4(n+1) \leq h+1 \leq \log_2(n+1)$.

NOTES

- ♦ Les réponses aux questions doivent être claires et justifiées.
- ♦ Pour chaque algorithme (ou fonction en C), vous aurez soin d'expliquer les différentes actions de l'algorithme ainsi que le passage des paramètres.
- ♦ **Chaque algorithme (ou fonction en C) supplémentaire utilisé doit être défini.**

*****Annexes*****

On suppose connu les algorithmes suivants :

Structure Liste simplement chaînée :

```
Fonction donnee : Liste → TElement
Fonction suivant : Liste → Liste
Fonction initL : [] → Liste
Fonction estVideL : Liste → Booléenne
Fonction inserTete : TElement x Liste → Liste
Fonction inserQueue : TElement x Liste → Liste
Fonction suppTete : Liste → Liste
```

Structure TElementDL :

```
Fonction valeur : TElementDL → Entier
Fonction occurrence : TElementDL → Entier
```

Structure DListe :

```
Fonction donneeDL : DListe → TElementDL
Fonction suivantDL : DListe → DListe
Fonction precedentDL : DListe → DListe
Fonction initDL : [] → DListe
Fonction estVideDL : DListe → Booléenne
```

Structure MultiEnsemble

```
Fonction taille : MultiEnsemble → Entier
Fonction tete : MultiEnsemble → DListe
Fonction queue : MultiEnsemble → DListe
```

Structure Arbres234

```
Fonction nbFils : Arbres234 → Entier
Fonction iEmeDonnee: Entier x Arbres234 → TElement
Fonction iEmeFils: Entier x Arbres234 → Arbre234
Fonction inserElt : TElement x Arbres234 → Arbre234
```

Structure Arbres Binaires

```
Fonction donneeA : Arbre → TElementA
Fonction filsGauche : Arbre → Arbre
Fonction filsDroit : Arbre → Arbre
Fonction initA : [] → Arbre
Fonction estVideA : Arbre → Booléenne
```

Structure Arbres généraux

```
Fonction donnee : ArbreG → TElement
Fonction iEmeFils : Entier x ArbreG → ArbreG
Fonction initAG : [] → ArbreG
Fonction estVideAG : ArbreG → Bool
```